1. Problem and Solution

Problem: A group of lizards live in a sago palm, and eat in the monkey grass across a driveway. Two cats sleep in the driveway, but wake up periodically. If a cat sees more than 4 lizards crossing, it plays with them, and the program ends. The program creates 20 lizard threads and 2 cat threads. Each lizard repeatedly sleeps, crosses to the monkey grass to eat, and then crosses back and sleeps again. The crossing must be synchronized so that no more than 4 lizards are on it at once, while still allowing up to 4 to cross simultaneously.

Solution: A counting semaphore (called `driveway`) starts at 4, and represents the 4 driveway spots available. A lizard takes a spot with sem_wait, before crossing, and returns it with sem_post once safely across. A lizard that finds no free spot sleeps in sem_wait, so there is no busy wait. A mutex (called `countLock`) protects the crossing counter, which several lizard threads update, and the cat threads read.

2. Changes to Code

- Added *driveway* semaphore and *countLock* mutex as globals.
- In *lizardThread*, the loop calls *sleepNow*, *sago2MonkeyGrassIsSafe*, *crossSago2MonkeyGrass*, *madeIt2MonkeyGrass*, *eat*, *monkeyGrass2SagoIsSafe,* *crossMonkeyGrass2Sago*, and *madeIt2Sago*, in that order.
- The IsSafe functions both call sem_wait(&driveway) to take a driveway spot, blocking if none are free.
- In the Cross and MadeIt functions, changes to the crossing counter are now wrapped in *countLock*.
- In main(), it now initializes everything, as well as deletes everything, to prevent memory leaks.

3. Results

WORLDEND	Lizards	Cats	Max Number of Lizards Crossing	Lizards Safe?
30	20	2	4	Yes
60	20	2	4	Yes
180	20	2	4	Yes
30	10	1	4	Yes
30	30	3	4	Yes

4. Issues Encountered

One issue was memory leaks. The cats and lizards were being properly deleted, but the cat and lizard threads were not, so it was fixed by adding destructors to lizard and cat classes.